

Mini UNIX Shell — User Guide

NizShell

1 Introduction

This document describes the usage of the **Mini UNIX Shell**, a lightweight command interpreter implemented in C. The shell provides essential UNIX command-line functionalities such as executing programs, pipelines, redirection, background tasks, and built-in commands. It behaves similarly to a simplified Bash environment while keeping the design clean and educational.

Once launched, the shell displays:

```
nizshell>
```

2 Supported Features

The shell implements the standard features expected from a minimal command interpreter:

- Executing external commands via the system `PATH`.
- Argument parsing with support for quoted strings.
- Input and output redirection using `<` and `>`.
- Pipelines, including multi-stage pipelines.
- Background execution using `&`.
- Built-in commands: `cd`, `exit`, and `history`.
- Signal handling so that `Ctrl-C` interrupts children but does not terminate the shell.
- Automatic cleanup of zombie processes.

These features allow the user to interact with the shell in a manner consistent with standard UNIX environments.

3 How to Build and Run

Build

```
make
```

Run

```
./nizshell
```

4 Usage Instructions

4.1 Running Standard Commands

Any program available in the system `PATH` can be invoked:

```
nizshell> ls
nizshell> pwd
nizshell> echo hello world
```

4.2 Quoted Arguments

The tokenizer supports both single and double quotation marks:

```
nizshell> echo "hello world"
nizshell> echo 'this is a test'
```

4.3 Input Redirection

Reads command input from a file:

```
nizshell> sort < names.txt
```

4.4 Output Redirection

Writes command output to a file:

```
nizshell> ls > files.txt
```

4.5 Pipelines

The shell supports multi-stage pipelines:

```
nizshell> ls | wc -l
nizshell> cat file.txt | grep hello | sort | uniq
```

4.6 Background Execution

Commands can run without blocking the shell:

```
nizshell> long_task &
[bg pid 2451]
```

4.7 Built-in Commands

cd

```
nizshell> cd /home
```

history

```
nizshell> history
  1 ls
  2 pwd
  3 history

  exit
nizshell> exit
```

5 Error Handling

The shell handles incorrect inputs safely and avoids crashes. Example behaviours include:

- Handling unknown commands.
- Detecting missing redirection files.
- Preventing empty pipeline stages.
- Ignoring empty lines.
- Ensuring the shell remains alive on **Ctrl-C**.

Example:

```
nizshell> nosuchcmd
execvp: No such file or directory
```

6 Internal Design (High-Level Overview)

Although the implementation details are internal to the shell, the design follows the standard architecture of UNIX shells:

Tokenizer

Processes user input into:

- individual words,
- quoted strings,
- control symbols (**|**, **<**, **>**, **&**).

Parser

Constructs a chain of **Command** structures, each representing:

- an argument vector,
- optional input redirection,
- optional output redirection.

Supports multi-stage pipelines.

Executor

Executes commands via:

- `fork()` and `execvp()` for external programs,
- redirecting file descriptors for redirection,
- setting up pipes for pipelines,
- handling background tasks without blocking the shell.

Signal Management

- The shell ignores `SIGINT`, preventing accidental termination.
- Child processes receive default signal behaviour.
- A `SIGCHLD` handler reaps finished background processes.

7 Screenshots

Screenshots demonstrating functionality are included in the `screenshots/` directory, covering:

- Basic command execution
- Argument parsing
- Redirection
- Pipelines
- Background processes
- History usage
- Error handling

8 Testing

A comprehensive test suite is provided in the `tests/` directory. It includes tests for:

- commands,
- redirection,
- pipelines,
- background jobs,
- error cases.

Run the full test suite with:

```
bash tests.txt
```

9 Conclusion

The Mini UNIX Shell provides a functional and robust UNIX-style command environment. It supports all core features expected in a basic shell, while maintaining a clear and modular internal architecture. It serves as a practical demonstration of key systems programming concepts including process control, pipes, signals, and parsing.